

Overview of Flash attention

Subject: Flash attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

1.

The idea of encoder–decoder sequence transduction had been developed in the early 2010s; commonly cited as the originators that produced seq2seq are two concurrently published papers from 2014. A 380M-parameter model for machine translation uses two long short-term memories (LSTM). Its architecture consists of two parts. The encoder is an LSTM that takes in a sequence of tokens and turns it into a vector. The decoder is another LSTM that converts the vector into a sequence of tokens. Similarly, another 130M-parameter model used gated recurrent units (GRU) instead of LSTM. Later research showed that GRUs are neither better nor worse than LSTMs for seq2seq. These early seq2seq models had no attention mechanism, and the state vector is accessible only after the last word of the source text was processed. Although in theory such a vector retains the information about the whole original sentence, in practice the information is poorly preserved. This is because the input is processed sequentially by one recurrent network into a fixed-size output vector, which is then processed by another recurrent network into an output. If the input is long, then the output vector would not be able to contain all relevant information, degrading the output. As evidence, reversing the input sentence improved seq2seq translation. The RNN search model introduced an attention mechanism to seq2seq for machine translation to solve the bottleneck problem (of the fixed-size output vector), allowing the model to process long-distance dependencies more easily. The name is because it "emulates searching through a source sentence during decoding a translation". The relative performances were compared between global (that of RNN search) and local (sliding window) attention model architectures for machine translation, finding that mixed attention had higher quality than global attention, while local attention reduced translation time. In 2016, Google Translate was revamped to Google Neural Machine Translation, which replaced the previous model based on statistical machine translation. The new model was a seq2seq model where the encoder and the decoder were both 8 layers of bidirectional LSTM. It took nine months to develop, and it outperformed the statistical approach, which took ten years to develop.

2.

Sub-quadratic transformers Training transformer-based architectures can be expensive, especially for long inputs. Many methods have been developed to attempt to address the issue. In the image domain, Swin transformer is an efficient architecture that performs attention inside shifting windows. In the audio domain, SepTr decouples the attention in time and frequency domains. Long Range Arena (2020) is a standard benchmark for comparing the behavior of transformer architectures over long inputs.

3.

Tokenization As the transformer architecture natively consists of operations over numbers (matrix multiplications, dot products, activation functions) rather than over text, there must first be a mapping from any input text to some numerical representation. This happens in three steps. First, the input text is treated by a preprocessor, which performs both textual transformations and splits the text into coarse-grained segments called pretokens. The latter is referred to as pretokenization. Second, each pretoken is segmented further into tokens by a tokenizer that expects to only see pretokens output by its preprocessor. Each token it produces is a string of one or more characters belonging to a finite set of strings called the vocabulary V . Third, because the vocabulary is finite and known beforehand, each token can be assigned an integer identifier, and this mapping is applied to the sequence of tokens to represent any input text as a numerical sequence. Since this mapping is bijective, the output side can produce a sequence of integer identifiers which can then be turned back into tokens. After undoing some of the preprocessing, the result is again legible text. Training a tokenizer (sometimes referred to as vocabularization) means finding a suitable vocabulary V , but also learning how to use it, since any given string s of length $|s|$ has $2^{|s|-1}$ hypothetical segmentations, some of which containing segments that are not in the vocabulary. The most important hyperparameter during vocabularization is the vocabulary size $|V|$: when it is small, the learned vocabulary generally consists of characters and smaller strings, and words will be segmented into many tokens. At larger sizes, it becomes affordable to dedicate tokens to full words, although depending on the preprocessor and tokenizer, it is not necessarily the case that large vocabularies will always use the largest token(s) available to segment a word. Because tokens are not always full words, they may also be referred to as subwords and tokenization algorithms may be referred to as subword tokenizers. This is also to differentiate these systems from traditional terminology used in older information retrieval and natural language processing systems, where "tokenization" was used to denote what is today called "pretokenization" (very crudely: splitting into words). In tokenizers that produce tokens that are not part of the vocabulary, a special token that does belong to the vocabulary is used as a generic stand-in, written as "[UNK]" for "unknown". In principle, any string could be hidden by such an [UNK]. Indeed, in information retrieval, pretokenizers were themselves used as tokenizers (and also called "tokenizers") with a word-level vocabulary that contained an [UNK]. Commonly used subword tokenization algorithms are byte pair encoding (BPE) and the unigram language model (ULM), which each include a vocabularization algorithm and a dedicated segmentation algorithm. There also exist several segmentation algorithms that require no learning and can be applied given a vocabulary (produced by BPE or ULM, for example), like greedily recognising tokens in a pretoken by moving through it left-to-right. Well-known software implementations of subword tokenizers are Hugging Face's tokenizers Python package implemented in Rust, and the sentencepiece Python package implemented in C++. The latter package is named as such because one of its configuration options allows disabling the built-in pretokenizer, hence effectively making entire sentences a pretoken and thus having the tokenizer see entire sentences, rather than individual words.

4.

Each integer token identifier is converted into an embedding vector via a lookup table. Equivalently stated, it multiplies a one-hot representation of the token identifier by an embedding matrix M . For example, if the input token's identifier is 3, then the one-hot representation is $[0, 0, 0, 1, 0, 0, \dots]$, and its embedding vector is $\text{Embed}(3) = [0, 0, 0, 1, 0, 0, \dots] M$.

]M} The token embedding vectors are added to their respective positional encoding vectors (see below), producing the sequence of input vectors. The dimension of an embedding vector is called hidden size or embedding size and written as d_{emb} . This size is written as d_{model} in the original transformer paper.